Name: INJA LAKSHMI SASIDHAR REDDY

USN :23BTRCL193

SECTION A: ADVANCED SQL || Maximum Marks: 20

Tasks:

a. Display employee names, department names, and manager names using joins. (2 marks)

```
62
63
64
65  select e.EmployeeName,d.DepartmentName,d.Manager_name from Employees e inner join Departments d on e.EmployeeID=d.DepartmentID
66
```

Output:

Results | Messages

	EmployeeName	DepartmentName	Manager_name
1	javeed	IT	Ramesh
2	sailaja	Operations	Praveen
3	sandeep	Research	Rahul
4	teja	Development	Vikram
5	naeeem	Support	Anil
6	basha	Testing	Sunil
7	marina	Security	Rajesh
8	jarian	Legal	Manoj
9	prabhas	Administration	Sai Kumar
10	dikshita	Logistics	Venkat
11	harsha	Analytics	Naveen

b. Find employees earning more than their department's average salary. (2 marks

```
SELECT
    e.EmployeeID,
    e.EmployeeName,
    e.DepartmentID,
    e.Salary
FROM Employees e
WHERE e.Salary > (
    SELECT AVG(e2.Salary)
    FROM Employees e2
    WHERE e2.DepartmentID = e.DepartmentID
);
```

Output:

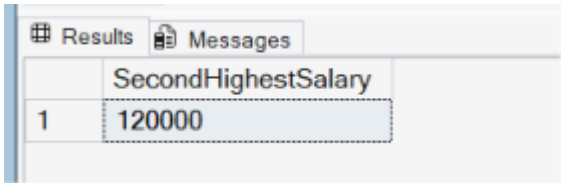


c. Find the second-highest distinct salary without using LIMIT, TOP, or FETCH

FIRST. (2 marks)

```sql
SELECT MAX(Salary) AS SecondHighestSalary
FROM Employees
WHERE Salary < (
    SELECT MAX(Salary)
    FROM Employees
);
```

Output:



Results | Messages

	SecondHighestSalary
1	120000

d. Rank employees by salary within each department and display the top three employees per department, retaining ties at the cutoff. (2 marks)

```sql
WITH RankedEmployees AS (
    SELECT
        EmployeeID,
        EmployeeName,
        DepartmentID,
        Salary,
        RANK() OVER (
            PARTITION BY DepartmentID
            ORDER BY Salary DESC
        ) AS SalaryRank
    FROM Employees
)
SELECT
    EmployeeID,
    EmployeeName,
    DepartmentID,
    Salary,
    SalaryRank
FROM RankedEmployees
WHERE SalaryRank <= 3
ORDER BY DepartmentID, Salary DESC;
```

Output:

116 || ORDER BY DepartmentID, Salary DESC;

82 % 16 0

Results Messages

	EmployeeID	EmployeeName	DepartmentID	Salary	SalaryRank
1	116	NULL	NULL	NULL	1
2	101	javeed	9631	70000	1
3	102	hussain	9632	60000	1
4	103	sasi	9633	65000	1
5	104	vasmi	9634	40000	1
6	105	pranavi	9635	67000	1
7	106	sailaja	9636	30000	1
8	107	sandeep	9637	80000	1
9	108	teja	9638	90000	1
10	109	naeeem	9639	100000	1
11	110	basha	9640	22000	1
12	111	marina	9641	78000	1
13	112	jarian	9642	49000	1
14	113	prabhas	9643	34000	1
15	114	dikshita	9644	120000	1
16	115	harsha	9645	760000	1

Question 2: Sales Analytics and Query Optimization (6 Marks)

a. Calculate total revenue for each product. (2 marks)

```
select productName,sum(unitprice) from sales group by productName
```

Output:

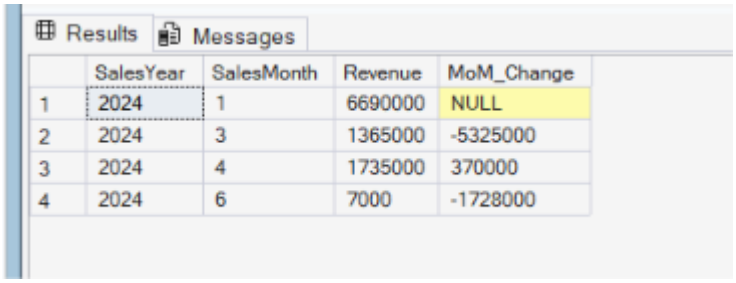
Results Messages

	productName	(No column name)
1	bottle	25000
2	charger	862000
3	gasstove	955000
4	jar	455000
5	laptop	710000
6	phone	770000
7	shoes	110000
8	slipers	355000

b. Calculate monthly revenue and month-over-month revenue change using a window function. (2 marks)

```sql
WITH MonthlyRevenue AS (
    SELECT
        YEAR(SalesDate) AS SalesYear,
        MONTH(SalesDate) AS SalesMonth,
        SUM(quantity * unitprice) AS Revenue
    FROM Sales
    GROUP BY YEAR(SalesDate), MONTH(SalesDate)
)
SELECT
    SalesYear,
    SalesMonth,
    Revenue,
    Revenue - LAG(Revenue) OVER (
        ORDER BY SalesYear, SalesMonth
    ) AS MoM_Change
FROM MonthlyRevenue
ORDER BY SalesYear, SalesMonth;
```

Output:



Results | Messages

	SalesYear	SalesMonth	Revenue	MoM_Change
1	2024	1	6690000	NULL
2	2024	3	1365000	-5325000
3	2024	4	1735000	370000
4	2024	6	7000	-1728000

c. Rewrite the following query to improve the potential for index usage and explain

why your version may perform better. (2 marks)

why your version may perform better. (2 marks)

```
SELECT *

FROM Employees

WHERE YEAR(JoiningDate) = 2025;
```

Assume JoiningDate may be indexed.

Output:

```
SELECT *

FROM Employees

WHERE JoiningDate >= '2025-01-01'

  AND JoiningDate <  '2026-01-01';
```

Question 3: SQL Troubleshooting (6 Marks)

How does an execution plan help identify expensive operations? (2 Marks)

- It identifies expensive operations such as table scans, index scans, joins, and sorting.

- It displays the estimated or actual cost, number of rows processed, and resource usage, helping us optimize slow queries.

b) Recommend an index strategy for DepartmentID and Salary. (2 Marks)

CREATE INDEX idx_employee_dept_salary

ON Employees (DepartmentID, Salary);

Reasoning:

- The index can improve queries filtering by both columns and queries filtering by DepartmentID alone.

c) Write a query to identify duplicate customer email addresses. (2 Marks)

SELECT Email, COUNT(*) AS DuplicateCount

FROM Customers

GROUP BY Email

HAVING COUNT(*) > 1;

SECTION B: LINUX || Maximum Marks: 15

SECTION B: LINUX — Maximum Marks: 15

Question 4: Log Analysis (5 Marks)

a. Find all lines containing ERROR (1 mark)

grep "ERROR" /var/log/application.log

b. Display the 20 most recent error entries in the file (1 mark)

grep "ERROR" /var/log/application.log | tail -n 20

c. Count the total number of error entries (1 mark)

grep -c "ERROR" /var/log/application.log

d. Extract error entries for a specified date into a separate file (1 mark)

grep "^2026-10-28.*ERROR" /var/log/application.log > /var/log/errors-2026-10-28.log

e. Explain how you would calculate the number of errors occurring in each hour (1 mark)

grep "ERROR" /var/log/application.log | awk '{print substr($2,1,2)}' | sort | uniq -c

Explanation: The grep command filters error entries, awk extracts the hour from the timestamp, sort groups identical hours together, and uniq -c counts the errors for each hour.

Question 5: Permissions and System Troubleshooting (5 Marks)

A Linux server hosts an application that processes confidential financial reports. Perform the following tasks:

a. Create a group named finance and a report file (1 mark)

sudo groupadd finance

sudo touch report.txt

b. Set permissions so the owner can read and write, the finance group can read, and other users have no access (1 mark)

sudo chown "$USER":finance report.txt

sudo chmod 640 report.txt

Explanation: Permission 640 gives the owner read/write (6), the finance group read-only (4), and others no permissions (0).

c. Identify the processes consuming the most CPU (1 mark)

ps aux --sort=-%cpu | head

d. Display memory usage and filesystem disk utilization (1 mark)

free -h

df -h

e. Find files larger than 100 MB under a specified application directory (1 mark)

find /path/to/application -type f -size +100M

Question 4: Log Analysis (5 Marks) Assume /var/log/application.log contains millions of log entries in the following format: 2026-10-28 10:15:23 ERROR payment-service Transaction failed Write Linux commands to:

a. Find all lines containing ERROR. (1 mark)

```
[ec2-user@ip-172-31-24-134 ~]$ grep "ERROR" /var/log/application.log
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:20:23 ERROR payment-service Payment declined
2026-10-28 11:05:23 ERROR auth-service Login failed
2026-10-28 12:30:23 ERROR database-service Connection failed
2026-10-28 12:45:23 ERROR payment-service Refund failed
```

b. Display the 20 most recent error entries in the file. (1 mark)

```
[ec2-user@ip-172-31-24-134 ~]$ 2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:20:23 ERROR payment-service Payment declined
2026-10-28 11:05:23 ERROR auth-service Login failed
2026-10-28 12:30:23 ERROR database-service Connection failed
2026-10-28 12:45:23 ERROR payment-service Refund failed
-bash: 2026-10-28: command not found
-bash: 2026-10-28: command not found
-bash: 2026-10-28: command not found
-bash: 2026-10-28: command not found
-bash: 2026-10-28: command not found
[ec2-user@ip-172-31-24-134 ~]$ grep "ERROR" /var/log/application.log | tail -n 20
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:20:23 ERROR payment-service Payment declined
2026-10-28 11:05:23 ERROR auth-service Login failed
2026-10-28 12:30:23 ERROR database-service Connection failed
2026-10-28 12:45:23 ERROR payment-service Refund failed
```

c. Count the total number of error entries. (1 mark)

```
[ec2-user@ip-172-31-24-134 ~]$ grep -c "ERROR" /var/log/application.log
5
```

d. Extract error entries for a specified date into a separate file. (1 mark) e. Explain how you would calculate the number of errors occurring in each hour. (1 mark)

```
[ec2-user@ip-172-31-24-134 ~]$ grep "ERROR" /var/log/application.log | awk '{print substr($2,1,2)}' | sort | uniq -c
      2 10
      1 11
      2 12
[ec2-user@ip-172-31-24-134 ~]$ grep "ERROR" /var/log/application.log | awk '{print $1, substr($2,1,2)":00"}' | sort | uniq -c
      2 2026-10-28 10:00
      1 2026-10-28 11:00
      2 2026-10-28 12:00
[ec2-user@ip-172-31-24-134 ~]$ ^C
[ec2-user@ip-172-31-24-134 ~]$
```

Question 5: Permissions and System Troubleshooting (5 Marks) A Linux server hosts an application that processes confidential financial reports. Perform the following tasks:

a. Create a group named finance and a report file, assuming you have the required privileges. (1 mark)

```
[ec2-user@ip-172-31-24-134 ~]$ cat /tmp/errors_2026-10-28.log
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:20:23 ERROR payment-service Payment declined
2026-10-28 11:05:23 ERROR auth-service Login failed
```

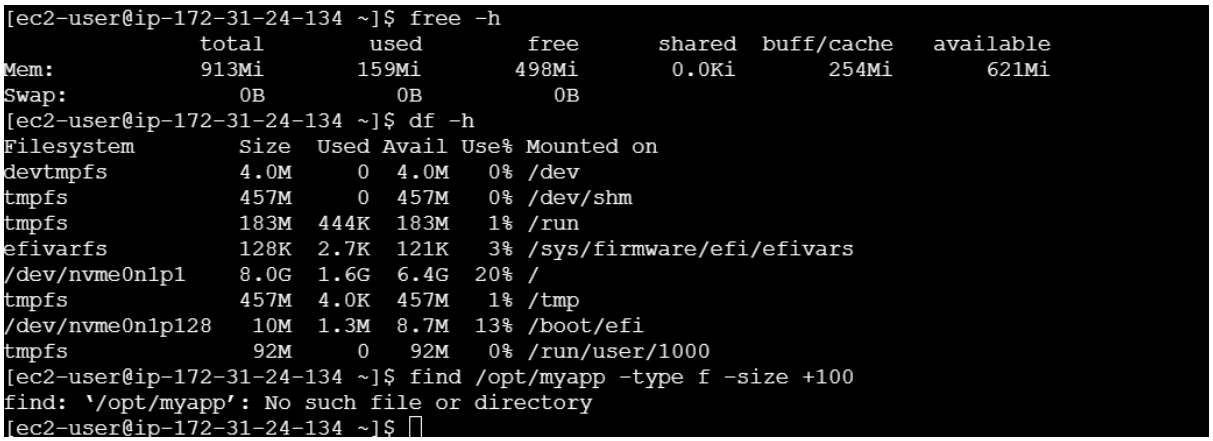
b. Set permissions so the owner can read and write, the finance group can read, and other users have no access. (1 mark)

```
top - 06:53:06 up 43 min,  1 user,  load average: 0.00, 0.00, 0.00
Tasks: 103 total,   1 running, 102 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :    913.1 total,    498.6 free,    160.0 used,    254.6 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    621.4 avail Mem
```

c. Identify the processes consuming the most CPU. (1 mark) d. Display memory usage and filesystem disk utilization. (1 mark)

```
    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0  173928  17868  11196 S   0.0   1.9   0:01.09 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     10 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     12 root      20   0       0      0      0 I   0.0   0.0   0:00.10 kworker/u8:0-flush-259:0
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 S   0.0   0.0   0:00.02 ksoftirqd/0
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.04 rcu_preempt
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_gp_kthread_worker
     18 root      rt   0       0      0      0 S   0.0   0.0   0:00.00 migration/0
     19 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
     21 root      rt   0       0      0      0 S   0.0   0.0   0:00.04 migration/1
     22 root      20   0       0      0      0 S   0.0   0.0   0:00.02 ksoftirqd/1
     24 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
     27 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kdevtmpfs
[ec2-user@ip-172-31-24-134 ~]$ free -h
               total        used        free      shared  buff/cache   available
```

e. Find files larger than 100 MB under a specified application directory. (1 mark)



SECTION C: PYSPARK DATA ENGINEERING || Maximum Marks: 20 Question 7: PySpark DataFrame Operations (6 Marks)

Create a PySpark DataFrame containing the following columns:

- EmployeeID
- EmployeeName
- Department
- Salary
- JoiningDate

Insert at least 15 sample records. Write and execute a PySpark program to:

```
from pyspark.sql.functions import col, avg, count
from datetime import datetime

spark = SparkSession.builder \
    .appName("EmployeeDataProcessing") \
    .getOrCreate()

# 2. Define schema and sample data (15 records)
data = [
    (1, "Sasidhar reddy", "IT", 65000, "2021-06-15"),
    (2, "Priya", "HR", 55000, "2020-03-12"),
    (3, "Rahul", "Finance", 72000, "2019-11-01"),
    (4, "Teja", "IT", 48000, "2022-01-10"),
    (5, "Javeed", "Marketing", 61000, "2021-08-25"),
    (6, "Sneha Reddy", "HR", 62000, "2018-05-20"),
    (7, "Vikram", "Finance", 58000, "2020-10-15"),
    (8, "Jabi", "Marketing", 45000, "2023-02-18"),
    (9, "Das", "IT", 80000, "2017-04-05"),
    (10, "Sandeep", "HR", 50000, "2022-09-01"),
    (11, "Sanjay", "Finance", 95000, "2016-12-12"),
    (12, "Divya", "IT", 63000, "2021-11-30"),
    (13, "Swrana", "Marketing", 70000, "2019-07-22"),
    (14, "Joshi", "HR", 67000, "2021-02-14"),
    (15, "Harsha", "Finance", 52000, "2023-05-05")
]

columns = ["EmployeeID", "EmployeeName", "Department", "Salary", "JoiningDate"]
```

```
+----------+--------------+----------+------+-----------+
|EmployeeID|  EmployeeName|Department|Salary|JoiningDate|
+----------+--------------+----------+------+-----------+
|         1|Sasidhar reddy|        IT| 65000| 2021-06-15|
|         2|         Priya|        HR| 55000| 2020-03-12|
|         3|         Rahul|   Finance| 72000| 2019-11-01|
|         4|          Teja|        IT| 48000| 2022-01-10|
|         5|        Javeed| Marketing| 61000| 2021-08-25|
|         6|   Sneha Reddy|        HR| 62000| 2018-05-20|
|         7|        Vikram|   Finance| 58000| 2020-10-15|
|         8|          Jabi| Marketing| 45000| 2023-02-18|
|         9|           Das|        IT| 80000| 2017-04-05|
|        10|       Sandeep|        HR| 50000| 2022-09-01|
|        11|        Sanjay|   Finance| 95000| 2016-12-12|
|        12|         Divya|        IT| 63000| 2021-11-30|
|        13|        Swrana| Marketing| 70000| 2019-07-22|
|        14|         Joshi|        HR| 67000| 2021-02-14|
|        15|        Harsha|   Finance| 52000| 2023-05-05|
+----------+--------------+----------+------+-----------+
```

a. Display the schema and first five records. (1 mark)

```
=== a. Display the schema and first five records ===
root
 |-- EmployeeID: long (nullable = true)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: date (nullable = true)

+----------+--------------+----------+------+-----------+
|EmployeeID|  EmployeeName|Department|Salary|JoiningDate|
+----------+--------------+----------+------+-----------+
|         1|Sasidhar reddy|        IT| 65000| 2021-06-15|
|         2|         Priya|        HR| 55000| 2020-03-12|
|         3|         Rahul|   Finance| 72000| 2019-11-01|
|         4|          Teja|        IT| 48000| 2022-01-10|
|         5|        Javeed| Marketing| 61000| 2021-08-25|
+----------+--------------+----------+------+-----------+
only showing top 5 rows
```

b. Filter employees whose monthly salary is greater than ₹60,000. (1 mark)

```
+----------+--------------+----------+------+-----------+
|EmployeeID|  EmployeeName|Department|Salary|JoiningDate|
+----------+--------------+----------+------+-----------+
|         1|Sasidhar reddy|        IT| 65000| 2021-06-15|
|         3|         Rahul|   Finance| 72000| 2019-11-01|
|         5|        Javeed| Marketing| 61000| 2021-08-25|
|         6|   Sneha Reddy|        HR| 62000| 2018-05-20|
|         9|           Das|        IT| 80000| 2017-04-05|
|        11|        Sanjay|   Finance| 95000| 2016-12-12|
|        12|         Divya|        IT| 63000| 2021-11-30|
|        13|        Swrana| Marketing| 70000| 2019-07-22|
|        14|         Joshi|        HR| 67000| 2021-02-14|
+----------+--------------+----------+------+-----------+
```

c. Add an AnnualSalary column calculated from the monthly salary. (1 mark)

```
+----------+--------------+----------+------+-----------+------------+
|EmployeeID|  EmployeeName|Department|Salary|JoiningDate|AnnualSalary|
+----------+--------------+----------+------+-----------+------------+
|         1|Sasidhar reddy|        IT| 65000| 2021-06-15|      780000|
|         2|         Priya|        HR| 55000| 2020-03-12|      660000|
|         3|         Rahul|   Finance| 72000| 2019-11-01|      864000|
|         4|          Teja|        IT| 48000| 2022-01-10|      576000|
|         5|        Javeed| Marketing| 61000| 2021-08-25|      732000|
+----------+--------------+----------+------+-----------+------------+
```

d. Group employees by department and calculate the average salary and employee count. (2marks)

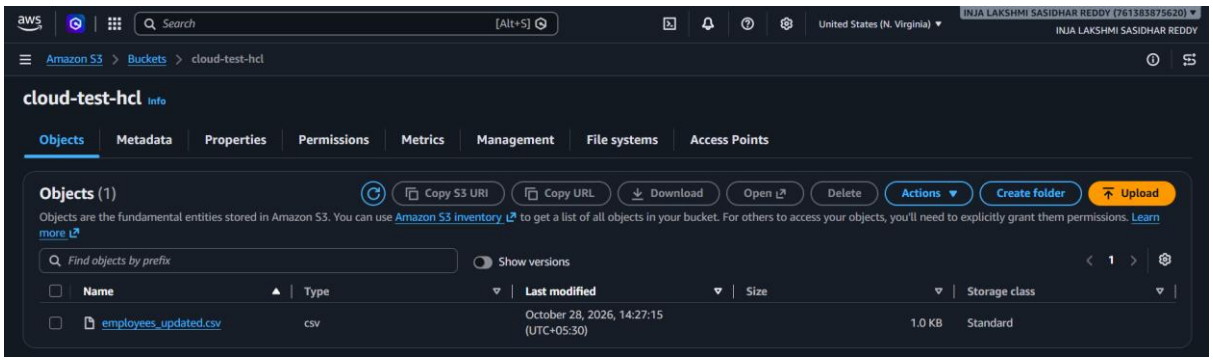
```
+----------+------------------+-------------+
|Department|     AverageSalary|EmployeeCount|
+----------+------------------+-------------+
|        HR|           58500.0|            4|
|   Finance|           69250.0|            4|
| Marketing|58666.666666666664|            3|
|        IT|           64000.0|            4|
+----------+------------------+-------------+
```

e. Display the five highest-paid employees. (1 mark)

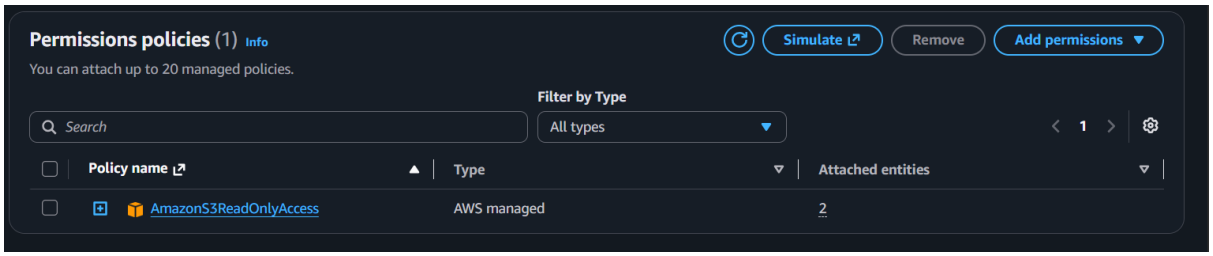
```
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|        11|      Sanjay|   Finance| 95000| 2016-12-12|
|         9|         Das|        IT| 80000| 2017-04-05|
|         3|       Rahul|   Finance| 72000| 2019-11-01|
|        13|      Swrana| Marketing| 70000| 2019-07-22|
|        14|       Joshi|        HR| 67000| 2021-02-14|
+----------+------------+----------+------+-----------+
```

Question 10: S3 and IAM Security

a. Create an S3 bucket with Block Public Access enabled and upload a sample file.



b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets.



c. Explain how you would verify that an unauthorized user cannot read the file.

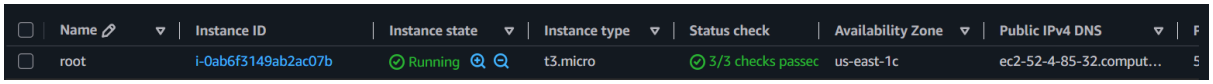
* When user trys to open file, account that does not have required permissions.it shows access is denied and user is not able open the file.

d. Explain how encryption at rest protects the stored data.

it stores data in encrypted form on disk. If an attacker get access to storage, the data remains unreadable with out correct decription word.

Question 11: EC2 Troubleshooting and CloudWatch

a. Check the EC2 instance state and status checks.



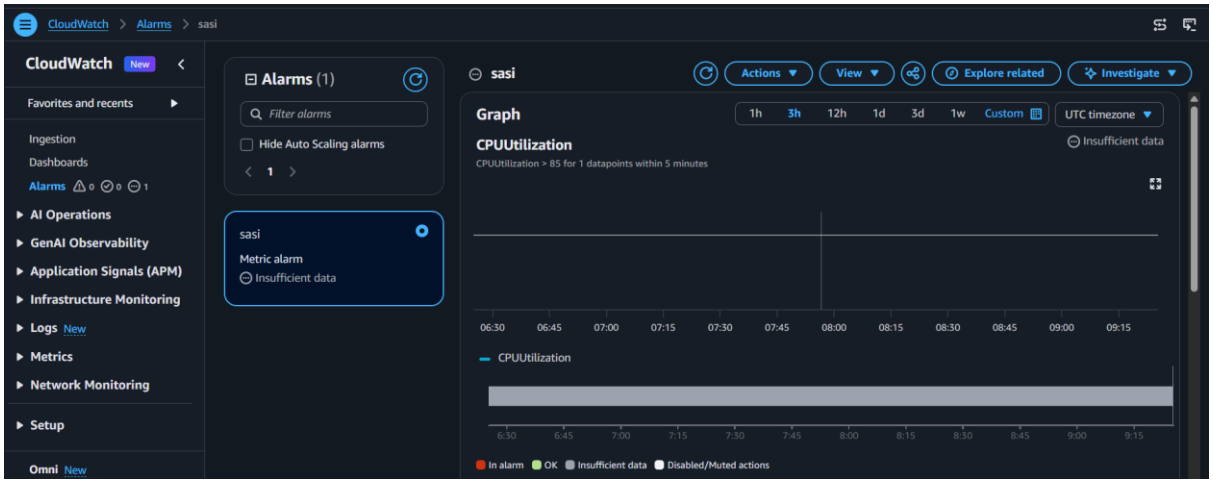
b. Explain how security groups and route tables can affect application connectivity.

Security groups filter traffic at the EC2 instance level, while route tables define the network path. Together, they ensure that only authorized traffic reaches the right destination.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period.

To configure a CloudWatch alarm for CPU utilization exceeding 85%, you need to define a metric, set a threshold, and specify actions.

Cloud watch -> create a alarm -> select cpu utilization ->define threshold and evaluation ->configure alarm actions -> Name and create alarm.



d. Identify one method for investigating the root cause of high CPU utilization.

One effective method for investigating the root cause of high CPU utilization is to analyze detailed performance metrics and logs from the instance.

Cloud watch -> detailed monitoring -> instance level investigation -> network and storage

nbuoQuestion 12: Event-Driven Processing (5 Marks)

a. Identify a suitable service for executing the processing code.

Aws Lambda is the best choice. It allows you to run code without provisioning servers, scales automatically, and integrates tightly with S3 events.

b. Explain how an S3 upload can trigger processing.

You configure an S3 Event Notification on the bucket. When a new CSV file is uploaded (e.g., PUT event), S3 sends an event to Lambda, which automatically invokes the function to validate and process the file.

c. Explain the IAM permissions needed to read the input file and write the results.

s3:GetObject permission to read the uploaded CSV file from the input bucket.

s3:PutObject permission to write the processed results to the output bucket or location. Optionally, s3:ListBucket if listing objects is required.

d. Explain how errors and processing logs can be captured.

Amazon CloudWatch Logs automatically captures Lambda execution logs, including errors and custom logging statements.

You can also configure CloudWatch Alarms or DLQs (Dead Letter Queues) with Amazon SQS/SNS to handle failed processing events.

e. Explain how to prevent duplicate processing or recursive triggering.

Use separate input and output buckets (or distinct prefixes) so that writing results does not trigger the same Lambda again.

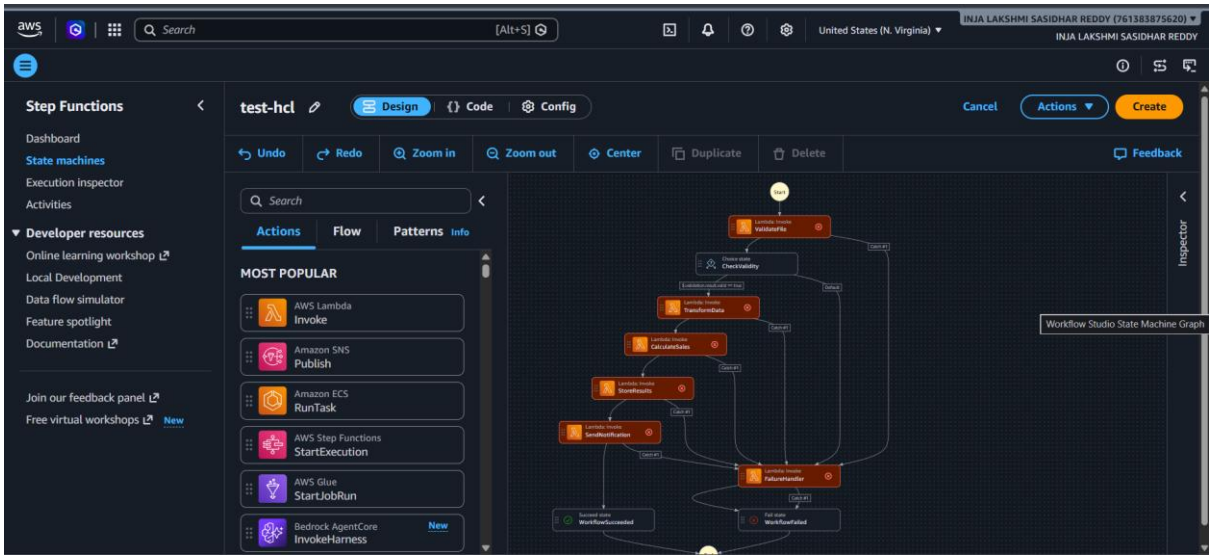
Add object metadata or DynamoDB tracking to ensure each file is processed only once.

Configure event filters in S3 to trigger only on specific prefixes or suffixes (e.g., .csv).

Question 13: State Machine Design

a. Identify suitable states for validation, processing, storage, and notification

Suitable States: Use Task states to run AWS Lambda functions or AWS SDK integrations for validation, data transformation/calculation, S3 upload, and SNS notification; optionally add a Choice state after validation for branching and a Fail or Succeed state at termination points.



b. Explain how a Choice state can route invalid files to a failure path and valid files to processing.

Choice State Routing: A Choice state inspects a boolean or status field in the validation output (e.g., $.isValid == true) to route valid files to the processing Task state and invalid files via Default or matching condition directly to a Fail state.

c. Explain how Retry and Catch can handle transient failures and permanent errors.

Retry vs. Catch: Retry automatically re-executes a failed state with configurable intervals and backoffs to overcome temporary glitches like network timeouts, while Catch intercepts unrecoverable or persistent errors to cleanly divert execution to a fallback or error-handling state.

d. Explain how the state machine can invoke Lambda functions or other supported AWS services.

Invoking Lambda & AWS Services: The state machine invokes services using a Task state by specifying the target service's ARN in the "Resource" field (such as a Lambda ARN or an optimized service integration like arn:aws:states:::sns:publish)

e. Explain how execution history helps troubleshoot a failed workflow.

Execution History Troubleshooting: The execution history logs a chronological step-by-step audit of state entries, exits, timestamps, exact input/output payloads, and detailed error tracebacks to quickly pinpoint the exact failure point.

Question 14: Step Functions Implementation

a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary.

```
{
  "Comment": "Sales Processing Workflow",
  "StartAt": "ValidateFile",
  "States": {
    "ValidateFile": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:ap-southeast-2:323477258763:function:ValidateFile",
      "Next": "TransformData"
    },
    "TransformData": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:ap-southeast-2:323477258763:function:TransformData",
      "Next": "StoreResults"
    },
    "StoreResults": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:ap-southeast-2:323477258763:function:StoreResults",
      "End": true
    }
  }
}
```

Sequential execution: AWS Step Functions executes ValidateFile, TransformData, and LoadResults in sequence using Task states.

b. Configure a retry policy for the transformation task with an appropriate error category, retry interval, and maximum attempts.

Validation: A Choice state checks validation results. If the file is invalid, the workflow stops using a Fail state.

c. Configure a Catch transition for the loading task that routes to an error-handling state.

Retry: The TransformData task uses Retry with IntervalSeconds: 2, MaxAttempts: 3, and BackoffRate: 2 to handle temporary failures.

d. Explain how data from one task is passed to the next task.

Catch: The LoadResults task uses Catch to route permanent errors to RecordFailure for logging.

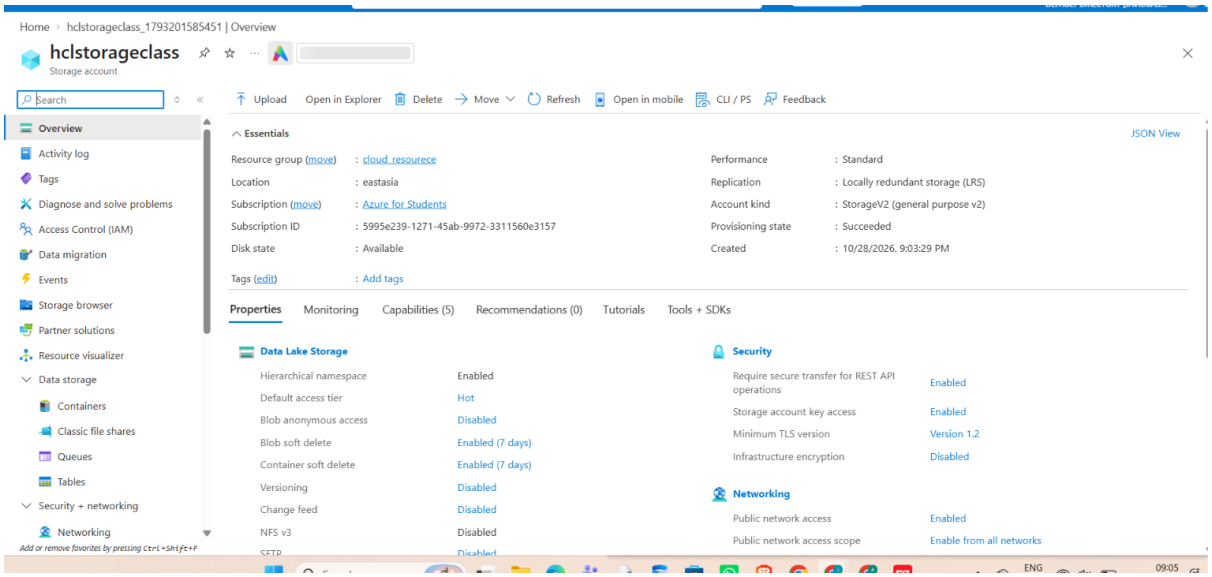
e. Explain how you would test successful and failed executions.

Data passing and testing: Payload.$ passes input between tasks, while ResultPath stores task outputs. Execution history helps debug errors and verify successful or failed runs.

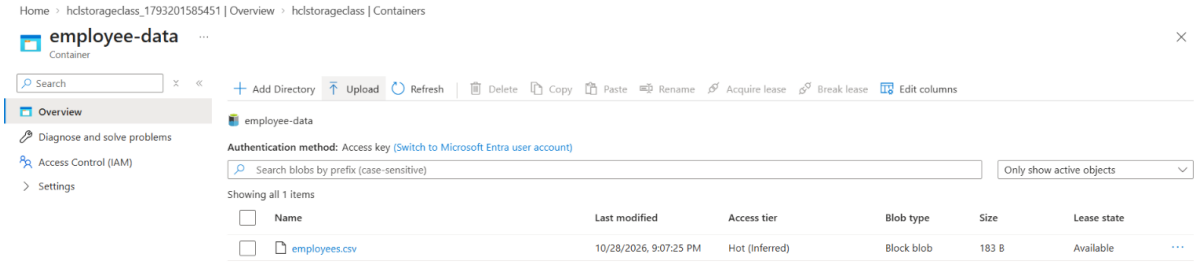
SECTION F: MICROSOFT AZURE || Maximum Marks: 15

Question 15: Azure Data Lake Storage and Security (5 Marks) A company needs to store confidential business data in Azure for analytics. Perform or document the following:

a. Configure or describe a storage account with hierarchical namespace enabled for Azure Data Lake Storage Gen2. (1 mark)



b. Create a filesystem or container and upload a sample dataset. (1 mark)



c. Explain how Microsoft Entra ID and Azure role-based access control can manage access to the data. (2 marks)

- Microsoft Entra ID: Authenticates users and verifies their identities.
- Azure RBAC: Assigns permissions to users or groups based on their roles.
- Storage Blob Data Reader: Allows users to read and download data.
- Storage Blob Data Contributor: Allows users to read, upload, and modify data.

d. Explain how you would verify that unauthorized users cannot access the dataset.(2 marks).

- Assign access only to authorized users.

- Sign in as an unauthorized user and try to access or download the CSV file.
- Verify that access is denied with an authorization error.
- Check Azure Monitor or diagnostic logs, if configured, for access attempts.

Question 16: Azure VM Troubleshooting

a. **Verify the VM's provisioning and power states**.

- In the Azure Portal, navigate to Virtual Machines and select the VM. Check:
- Provisioning state: Succeeded, Failed or another reported state.
- Power state: Running, Stopped, Deallocated or another reported state.

b. Explain how Network Security Group rules can prevent connectivity.

- NSGs contain inbound and outbound rules that permit or deny network traffic. A missing SSH rule for TCP port 22 or RDP rule for TCP port 3389 can prevent connections. Rules are evaluated according to priority.

c. Explain how subnet and routing configuration affect connectivity.

- An incorrect subnet configuration, user-defined route, route table or next-hop setting can prevent traffic from reaching the VM. Verify the effective routes, subnet association, network interface configuration and any required gateway or peering connections.

d. Recommend a secure troubleshooting method that avoids unnecessarily exposing the VM to the public internet.

- Use Azure Bastion to connect to the VM over SSH or RDP through the Azure Portal without assigning the VM a public IP address.
- Additional checks:
- Use Network Watcher connection troubleshooting.
- Review NSG effective security rules and effective routes.
- Check VM boot diagnostics and serial console where supported.
- Avoid opening SSH or RDP to the entire internet.

Question 17: Azure Data Integration

a.Select an appropriate service to orchestrate data movement.

- **Use Azure Data Factory (ADF) to create and orchestrate a data pipeline that automates the movement and processing of daily customer transaction CSV files**

b. Copy the CSV file into Azure Data Lake Storage Gen2 (1 mark)

Create an Azure Data Factory pipeline with a Copy Data activity. Configure the CSV file as the source and Azure Data Lake Storage Gen2 as the destination. Set up the required linked services and authentication to transfer the file securely.

c. Describe how to organize files by ingestion date. (1 mark)

ADF dynamic content can generate the folder path using the ingestion date, making files easy to locate and manage.

d. Handle malformed records and missing values (1 mark)

Use Azure Databricks or Azure Data Factory Mapping Data Flows to validate and transform the data. Send malformed records to a separate error folder, handle missing values by applying suitable defaults or null values, and remove duplicates where necessary.

e. Monitor pipeline execution and validate data completeness (1 mark)

Use Azure Data Factory Monitor to track pipeline runs, failures, and execution duration. Validate data completeness by comparing source and destination record counts, checking required fields, and logging rejected records. Configure Azure Monitor alerts for pipeline failures.